Python Basics: A Comprehensive Guide for Beginners

Welcome to Python Programming!

This notebook covers all fundamental Python programming concepts with detailed explanations, code examples, and output demonstrations.

Target Audience: Beginners with no prior programming experience

Created: December 29, 2025

Table of Contents

1. Variables and Data Types
2. Numbers and Arithmetic Operations
3. Strings and String Operations
4. Lists
5. Tuples
6. Dictionaries
7. Sets
8. Booleans and Comparison Operators
9. Conditional Statements (if, elif, else)
10. Loops (for and while)
11. Functions
12. Input and Output
13. Type Conversion
14. String Formatting
15. Working with Files
16. Error Handling
17. List Comprehensions
18. Lambda Functions

---

1. Variables and Data Types {#variables-and-data-types}

Concept Explanation

A **variable** is a container for storing data values. Python automatically detects the data type based on the value assigned.

Main Data Types:

- **int**: Integer (whole numbers)

- **float**: Floating-point numbers (decimals)
- **str**: Strings (text)
- **bool**: Boolean (True/False)
- **list**: Collection of items
- **tuple**: Immutable collection
- **dict**: Key-value pairs
- **set**: Unique items

```
# Example 1: Creating Variables and Checking Data Types
name = "Alice"          # String variable
age = 25               # Integer variable
height = 5.6           # Float variable
is_student = True      # Boolean variable

print("Variable Values and Their Types:")
print(f"Name: {name}, Type: {type(name)}")
print(f"Age: {age}, Type: {type(age)}")
print(f"Height: {height}, Type: {type(height)}")
print(f"Is Student: {is_student}, Type: {type(is_student)}")
```

Output Explanation:

The `type()` function returns the data type of a variable. Notice how Python automatically assigned the correct type based on the value we provided.

```
# Example 2: Variable Assignment and Reassignment
count = 10
print(f"Initial count: {count}")

count = 20  # Reassigning the variable
print(f"Updated count: {count}")

# Multiple assignment
x, y, z = 1, 2, 3
print(f"\nMultiple Assignment: x={x}, y={y}, z={z}")
```

---

2. Numbers and Arithmetic Operations {#numbers-and-arithmetic-operations}

Concept Explanation

Python supports various arithmetic operations: addition (+), subtraction (-), multiplication (*), division (/), floor division (//), modulus (%), and exponentiation (**).

```
# Example 1: Basic Arithmetic Operations
a = 15
b = 4
```

```
print("Arithmetic Operations:")
print(f"Addition: {a} + {b} = {a + b}")
print(f"Subtraction: {a} - {b} = {a - b}")
print(f"Multiplication: {a} * {b} = {a * b}")
print(f"Division: {a} / {b} = {a / b}")
print(f"Floor Division: {a} // {b} = {a // b}")
print(f"Modulus (Remainder): {a} % {b} = {a % b}")
print(f"Exponentiation: {a} ** {b} = {a ** b}")
```

Output Explanation:

- **Division (/)** always returns a float, even if both numbers are integers
- **Floor Division (//)** returns an integer (rounds down)
- **Modulus (%)** returns the remainder
- **Exponentiation ()**** raises to a power

```
# Example 2: Order of Operations (PEMDAS)
result1 = 10 + 5 * 2
result2 = (10 + 5) * 2

print("Order of Operations:")
print(f"10 + 5 * 2 = {result1}  (Multiplication first)")
print(f"(10 + 5) * 2 = {result2}  (Parentheses first)")
```

```
# Example 3: Augmented Assignment Operators
x = 10
print(f"Initial value: x = {x}")

x += 5  # Same as x = x + 5
print(f"After x += 5: x = {x}")

x -= 3  # Same as x = x - 3
print(f"After x -= 3: x = {x}")

x *= 2  # Same as x = x * 2
print(f"After x *= 2: x = {x}")

x //= 4  # Same as x = x // 4
print(f"After x //= 4: x = {x}")
```

---

3. Strings and String Operations {#strings-and-string-operations}

Concept Explanation

Strings are sequences of characters enclosed in quotes (single or double). They support various operations like concatenation, repetition, and slicing.

```
# Example 1: String Creation and Basic Operations
str1 = "Hello"
```

```
str2 = "World"

print("String Operations:")
print(f"String 1: {str1}")
print(f"String 2: {str2}")
print(f"Concatenation: {str1} + {str2} = {str1 + ' ' + str2}")
print(f"Repetition: {str1} * 3 = {str1 * 3}")
print(f"Length: len('{str1}') = {len(str1)}")
```

```
# Example 2: String Indexing and Slicing
text = "Python"

print(f"String: {text}")
print(f"Index positions: 0={text[0]}, 1={text[1]}, 2={text[2]}, 3={text[3]}, 4={text[4]
print(f"\nNegative indexing: -1={text[-1]}, -2={text[-2]}, -3={text[-3]}")
print(f"\nSlicing:")
print(f"text[0:3] = {text[0:3]}  (characters from index 0 to 2)")
print(f"text[2:5] = {text[2:5]}  (characters from index 2 to 4)")
print(f"text[:3] = {text[:3]}    (first 3 characters)")
print(f"text[3:] = {text[3:]}    (from index 3 to end)")
print(f"text[::2] = {text[::2]}  (every 2nd character)")
```

Output Explanation:

- String indexing starts at 0
- Negative indices count from the end (-1 is the last character)
- Slicing syntax: `string[start:end:step]` where end is exclusive

```
# Example 3: String Methods
sentence = "hello world python"

print(f"Original: {sentence}")
print(f"Upper: {sentence.upper()}")
print(f"Lower: {sentence.lower()}")
print(f"Capitalize: {sentence.capitalize()}")
print(f"Title Case: {sentence.title()}")
print(f"Replace 'python' with 'java': {sentence.replace('python', 'java')}")
print(f"Split into list: {sentence.split()}")
print(f"Count 'o': {sentence.count('o')}")
print(f"Starts with 'hello': {sentence.startswith('hello')}")
print(f"Ends with 'python': {sentence.endswith('python')}")
```

---

4. Lists {#lists}

Concept Explanation

A **list** is a mutable (changeable) collection of items. Lists can contain different data types and are created using square brackets `[]`.

```
# Example 1: Creating and Accessing Lists
fruits = ["apple", "banana", "orange", "mango"]
```

```
numbers = [10, 20, 30, 40, 50]
mixed = [1, "hello", 3.14, True]

print("List Creation:")
print(f"Fruits: {fruits}")
print(f"Numbers: {numbers}")
print(f"Mixed Types: {mixed}")
print(f"\nAccessing Elements:")
print(f"First fruit: {fruits[0]}")
print(f"Last fruit: {fruits[-1]}")
print(f"Length of fruits list: {len(fruits)}")
```

```
# Example 2: List Methods - Adding and Removing Elements
colors = ["red", "blue", "green"]

print(f"Original list: {colors}")

colors.append("yellow")
print(f"After append('yellow'): {colors}")

colors.insert(1, "purple")
print(f"After insert(1, 'purple'): {colors}")

colors.remove("purple")
print(f"After remove('purple'): {colors}")

popped = colors.pop()  # Removes last element
print(f"After pop(): {colors}, Popped element: {popped}")

colors.extend(["pink", "orange"])
print(f"After extend(['pink', 'orange']): {colors}")
```

```
# Example 3: List Slicing and Operations
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

print(f"Original list: {numbers}")
print(f"\nSlicing:")
print(f"numbers[2:5] = {numbers[2:5]}")
print(f"numbers[::2] = {numbers[::2]}  (every 2nd element)")
print(f"numbers[::-1] = {numbers[::-1]}  (reversed)")
print(f"\nList Operations:")
print(f"Sum: {sum(numbers)}")
print(f"Max: {max(numbers)}")
print(f"Min: {min(numbers)}")
print(f"Sorted: {sorted(numbers)}")
print(f"Sorted (reverse): {sorted(numbers, reverse=True)}")
```

```
# Example 4: Iterating Through Lists
students = ["Alice", "Bob", "Charlie"]

print("Using a simple for loop:")
for student in students:
    print(f"  - {student}")

print("\nUsing enumerate (with index):")
for index, student in enumerate(students, start=1):
    print(f"  {index}. {student}")
```

5. Tuples {#tuples}

Concept Explanation

A **tuple** is an immutable (unchangeable) collection of items. Tuples are created using parentheses `()`. Once created, elements cannot be modified.

```
# Example 1: Creating and Accessing Tuples
coordinates = (10, 20, 30)
single_item = ("hello",)  # Note: comma is required for single-item tuple
mixed_tuple = (1, "two", 3.0, True)

print("Tuple Creation:")
print(f"Coordinates: {coordinates}")
print(f"Single item tuple: {single_item}")
print(f"Mixed tuple: {mixed_tuple}")
print(f"\nAccessing Elements:")
print(f"First element: {coordinates[0]}")
print(f"Last element: {coordinates[-1]}")
print(f"Length: {len(coordinates)}")
```

```
# Example 2: Tuple Unpacking
rgb = (255, 128, 0)
r, g, b = rgb

print(f"Tuple: {rgb}")
print(f"Unpacked: r={r}, g={g}, b={b}")

# Tuple methods
print(f"\nTuple Methods:")
numbers = (1, 2, 3, 2, 4, 2)
print(f"Numbers tuple: {numbers}")
print(f"Count of 2: {numbers.count(2)}")
print(f"Index of 3: {numbers.index(3)}")
```

Output Explanation:

- Tuples are immutable, so you cannot modify their elements
- Tuples are faster than lists for fixed data
- Useful for returning multiple values from functions

---

6. Dictionaries {#dictionaries}

Concept Explanation

A **dictionary** is an unordered collection of key-value pairs. Dictionaries are created using curly braces `{}` with keys and values separated by colons.

```python
# Example 1: Creating and Accessing Dictionaries
person = {
    "name": "John",
    "age": 30,
    "city": "New York",
    "email": "john@example.com"
}

print("Dictionary Creation:")
print(f"Person: {person}")
print(f"\nAccessing Values:")
print(f"Name: {person['name']}")
print(f"Age: {person['age']}")
print(f"City: {person.get('city')}")
print(f"Default value for missing key: {person.get('phone', 'Not provided')}")
```

```python
# Example 2: Adding and Modifying Dictionary Items
student = {"name": "Alice", "grade": "A"}

print(f"Original: {student}")

student["age"] = 20  # Add new key-value pair
print(f"After adding age: {student}")

student["grade"] = "A+"  # Modify existing value
print(f"After updating grade: {student}")

student.update({"city": "Boston", "major": "Computer Science"})
print(f"After update(): {student}")
```

```python
# Example 3: Dictionary Methods
book = {"title": "Python Basics", "author": "John Doe", "pages": 300}

print(f"Dictionary: {book}")
print(f"\nDictionary Methods:")
print(f"Keys: {book.keys()}")
print(f"Values: {book.values()}")
print(f"Items: {book.items()}")
print(f"Number of items: {len(book)}")
print(f"'author' in book: {'author' in book}")
print(f"'price' in book: {'price' in book}")
```

```python
# Example 4: Iterating Through Dictionaries
scores = {"Math": 85, "Science": 90, "English": 88}

print("Iterating through dictionary:")
print("\nUsing items():")
for subject, score in scores.items():
    print(f"  {subject}: {score}")

print("\nUsing keys():")
for key in scores.keys():
    print(f"  {key}")

print("\nUsing values():")
for value in scores.values():
    print(f"  {value}")
```

7. Sets {#sets}

Concept Explanation

A **set** is an unordered collection of unique items. Sets are created using curly braces `{}` with individual items. Duplicates are automatically removed.

```
# Example 1: Creating Sets and Removing Duplicates
colors = {"red", "blue", "green", "red", "blue"}

print(f"Set with duplicates input: {{'red', 'blue', 'green', 'red', 'blue'}}")
print(f"Resulting set (duplicates removed): {colors}")
print(f"Type: {type(colors)}")

# Converting list to set to remove duplicates
numbers = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
unique_numbers = set(numbers)
print(f"\nList with duplicates: {numbers}")
print(f"Set (unique values): {unique_numbers}")
```

```
# Example 2: Set Operations
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

print(f"Set A: {set_a}")
print(f"Set B: {set_b}")
print(f"\nSet Operations:")
print(f"Union (A | B): {set_a | set_b}")
print(f"Intersection (A & B): {set_a & set_b}")
print(f"Difference (A - B): {set_a - set_b}")
print(f"Symmetric Difference (A ^ B): {set_a ^ set_b}")
```

```
# Example 3: Set Methods
fruits = {"apple", "banana", "cherry"}

print(f"Original set: {fruits}")

fruits.add("orange")
print(f"After add('orange'): {fruits}")

fruits.remove("banana")
print(f"After remove('banana'): {fruits}")

fruits.discard("grape")  # Doesn't raise error if item not found
print(f"After discard('grape'): {fruits}")

popped = fruits.pop()  # Removes arbitrary item
print(f"After pop(): {fruits}, Popped: {popped}")
```

8. Booleans and Comparison Operators {#booleans-and-comparison-operators}

Concept Explanation

Boolean values are `True` or `False`. **Comparison operators** compare two values and return a boolean result.

```
# Example 1: Comparison Operators
a = 10
b = 5

print("Comparison Operators:")
print(f"{a} == {b}: {a == b}  (equal)")
print(f"{a} != {b}: {a != b}  (not equal)")
print(f"{a} > {b}: {a > b}   (greater than)")
print(f"{a} < {b}: {a < b}   (less than)")
print(f"{a} >= {b}: {a >= b}  (greater than or equal)")
print(f"{a} <= {b}: {a <= b}  (less than or equal)")
```

```
# Example 2: Logical Operators (and, or, not)
print("Logical Operators:")
print(f"True and True: {True and True}")
print(f"True and False: {True and False}")
print(f"True or False: {True or False}")
print(f"False or False: {False or False}")
print(f"not True: {not True}")
print(f"not False: {not False}")

print("\nComplex Logical Expression:")
age = 25
has_license = True
is_available = (age >= 18) and (has_license)
print(f"Age: {age}, License: {has_license}")
print(f"Can drive: {is_available}")
```

```
# Example 3: Membership Operators (in, not in)
fruits = ["apple", "banana", "orange"]
word = "hello"

print("Membership Operators:")
print(f"'apple' in {fruits}: {'apple' in fruits}")
print(f"'grape' in {fruits}: {'grape' in fruits}")
print(f"'e' in '{word}': {'e' in word}")
print(f"'z' not in '{word}': {'z' not in word}")
```

---

9. Conditional Statements (if, elif, else) {#conditional-statements}

Concept Explanation

Conditional statements allow your program to make decisions based on different conditions. Python uses `if`, `elif`, and `else` keywords.

```
# Example 1: Simple if-else
age = 16

print("Example 1: Checking if eligible to vote")
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote yet.")
```

```
# Example 2: if-elif-else
score = 75

print("Example 2: Grade Assignment")
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print(f"Score: {score} -> Grade: {grade}")
```

```
# Example 3: Nested Conditions
print("Example 3: Nested Conditions")
temperature = 25
is_raining = False

if temperature > 20:
    print(f"Temperature is {temperature}°C - It's warm")
    if is_raining:
        print("  It's warm and rainy. Bring an umbrella.")
    else:
        print("  It's warm and sunny. Great day!")
else:
    print(f"Temperature is {temperature}°C - It's cold")
```

```
# Example 4: Ternary Operator (Conditional Expression)
print("Example 4: Ternary Operator")
age = 20
status = "Adult" if age >= 18 else "Minor"
print(f"Age: {age} -> Status: {status}")

number = 7
result = "Even" if number % 2 == 0 else "Odd"
print(f"Number: {number} -> {result}")
```

---

10. Loops (for and while) {#loops}

Concept Explanation

Loops allow you to execute a block of code multiple times. Python has two main types: `for` loops and `while` loops.

```
# Example 1: for loop with range()
print("Example 1: for loop with range()")
print("Numbers from 1 to 5:")
for i in range(1, 6):
    print(f"  i = {i}")

print("\nNumbers from 0 to 10 (step 2):")
for i in range(0, 11, 2):
    print(f"  i = {i}")
```

```
# Example 2: for loop with collections
print("Example 2: for loop with collections")
fruits = ["apple", "banana", "cherry"]

print("Iterating through list:")
for fruit in fruits:
    print(f"  - {fruit}")

print("\nIterating with index:")
for index, fruit in enumerate(fruits):
    print(f"  {index}: {fruit}")
```

```
# Example 3: while loop
print("Example 3: while loop")
count = 0

print("Counting from 0 to 4:")
while count < 5:
    print(f"  count = {count}")
    count += 1

print("\nUserInput simulation:")
password = ""
correct_password = "123456"
attempts = 0

# Simulating password check (fixed sequence)
print("Checking password...")
if password != correct_password:
    password = "123456"
    attempts = 1
    print(f"  Attempt {attempts}: Correct password entered!")
```

```
# Example 4: break and continue statements
print("Example 4: break and continue")

print("Using break (exit loop when condition is met):")
```

```
for i in range(1, 11):
    if i == 5:
        print(f"  Found 5! Breaking out of loop.")
        break
    print(f"  i = {i}")

print("\nUsing continue (skip to next iteration):")
for i in range(1, 6):
    if i == 3:
        print(f"  Skipping 3...")
        continue
    print(f"  i = {i}")
```

---

11. Functions {#functions}

Concept Explanation

A **function** is a reusable block of code that performs a specific task. Functions help organize code and make it more maintainable.

```
# Example 1: Simple Function
def greet():
    """This function prints a greeting message."""
    print("Hello! Welcome to Python")

print("Example 1: Simple Function")
greet()
```

```
# Example 2: Function with Parameters
def greet_person(name, age):
    """Greet a person with their name and age."""
    print(f"Hello {name}! You are {age} years old.")

print("Example 2: Function with Parameters")
greet_person("Alice", 25)
greet_person("Bob", 30)
```

```
# Example 3: Function with Return Value
def add(a, b):
    """Add two numbers and return the result."""
    return a + b

def multiply(a, b):
    """Multiply two numbers and return the result."""
    return a * b

print("Example 3: Function with Return Value")
sum_result = add(10, 5)
product_result = multiply(10, 5)

print(f"add(10, 5) = {sum_result}")
print(f"multiply(10, 5) = {product_result}")
```

```
# Example 4: Function with Default Parameters
def power(base, exponent=2):
    """Calculate base raised to the power of exponent."""
    return base ** exponent

print("Example 4: Function with Default Parameters")
print(f"power(5) = {power(5)}       (uses default exponent=2)")
print(f"power(5, 3) = {power(5, 3)} (uses exponent=3)")
```

```
# Example 5: Function Returning Multiple Values
def get_student_info():
    """Return student information as tuple."""
    name = "John"
    age = 20
    grade = "A"
    return name, age, grade  # Returns a tuple

print("Example 5: Multiple Return Values")
name, age, grade = get_student_info()
print(f"Name: {name}, Age: {age}, Grade: {grade}")
```

```
# Example 6: *args and **kwargs
def sum_all(*args):
    """Sum all arguments passed to the function."""
    total = 0
    for num in args:
        total += num
    return total

def print_info(**kwargs):
    """Print key-value pairs."""
    for key, value in kwargs.items():
        print(f"  {key}: {value}")

print("Example 6: *args and **kwargs")
print(f"sum_all(1, 2, 3, 4, 5) = {sum_all(1, 2, 3, 4, 5)}")
print(f"sum_all(10, 20, 30) = {sum_all(10, 20, 30)}")

print("\nprint_info with keyword arguments:")
print_info(name="Alice", age=25, city="Boston")
```

---

12. Input and Output {#input-and-output}

Concept Explanation

Input: Receiving data from the user using `input()` function

Output: Displaying data using `print()` function

```
# Example 1: print() function
print("Example 1: print() function")
print("Hello, World!")
print("Python", "is", "awesome")
```

```
print(1, 2, 3, sep="-", end="!\n")  # Custom separator and end
print()
```

```
# Example 2: f-strings (formatted strings)
print("Example 2: f-strings (Python 3.6+)")
name = "Alice"
age = 25
score = 95.5

print(f"My name is {name}")
print(f"I am {age} years old")
print(f"My score is {score:.1f}")
print(f"Next year I'll be {age + 1} years old")
```

```
# Example 3: input() function
print("Example 3: input() function")
print("Note: In Jupyter, this will prompt below the cell")

# Simulating input without interactive prompt
user_name = "Charlie"  # Simulating user input
print(f"Hello {user_name}!")
print(f"Nice to meet you, {user_name}!")
```

```
# Example 4: String formatting methods
print("Example 4: Different string formatting methods")
name = "Bob"
score = 85.567

# Method 1: % operator
print("Method 1: %s has scored %d%% in the exam" % (name, int(score)))

# Method 2: .format()
print("Method 2: {} has scored {:.2f} in the exam".format(name, score))

# Method 3: f-string
print(f"Method 3: {name} has scored {score:.2f} in the exam")
```

---

13. Type Conversion {#type-conversion}

Concept Explanation

Type conversion (or type casting) is converting a variable from one type to another.

```
# Example 1: Converting to int, float, and str
print("Example 1: Basic Type Conversions")

# String to int
str_num = "123"
int_num = int(str_num)
print(f"String '123' to int: {int_num}, Type: {type(int_num)}")

# String to float
str_decimal = "45.67"
```

```
float_num = float(str_decimal)
print(f"String '45.67' to float: {float_num}, Type: {type(float_num)}")

# Number to string
number = 100
str_from_num = str(number)
print(f"Number 100 to string: '{str_from_num}', Type: {type(str_from_num)}")
```

```
# Example 2: Converting to bool
print("Example 2: Converting to bool")
print(f"bool(0) = {bool(0)}")
print(f"bool(1) = {bool(1)}")
print(f"bool('') = {bool('')}")
print(f"bool('hello') = {bool('hello')}")
print(f"bool([]) = {bool([])}")
print(f"bool([1, 2, 3]) = {bool([1, 2, 3])}")
```

```
# Example 3: Converting Collections
print("Example 3: Converting Between Collections")

# List to tuple
my_list = [1, 2, 3, 4, 5]
my_tuple = tuple(my_list)
print(f"List {my_list} to tuple: {my_tuple}")

# String to list
text = "Python"
char_list = list(text)
print(f"String '{text}' to list: {char_list}")

# List to set (remove duplicates)
numbers = [1, 2, 2, 3, 3, 3, 4]
unique = set(numbers)
print(f"List {numbers} to set: {unique}")
```

---

14. String Formatting {#string-formatting}

Concept Explanation

String formatting is creating strings with dynamic values embedded in them. Python offers multiple methods.

```
# Example 1: f-strings (Modern approach)
print("Example 1: f-strings (Recommended)")
name = "Diana"
age = 28
salary = 65000.50

print(f"Name: {name}")
print(f"Age: {age} years old")
print(f"Salary: ${salary:,.2f}")
print(f"Next year: {age + 1}")
```

```
# Example 2: .format() method
print("Example 2: .format() method")
template = "Hello {}, you are {} years old"
print(template.format("Eve", 30))

template2 = "Hello {name}, you are {age} years old"
print(template2.format(name="Frank", age=35))
```

```
# Example 3: % operator (Old style)
print("Example 3: % operator (Old style)")
print("Hello %s, you are %d years old" % ("Grace", 22))
print("Pi is approximately %.2f" % 3.14159)
```

```
# Example 4: Formatting Specifications
print("Example 4: Formatting Specifications")
pi = 3.14159265
price = 19.5
count = 42

print(f"Pi rounded to 2 decimals: {pi:.2f}")
print(f"Pi rounded to 4 decimals: {pi:.4f}")
print(f"Price with currency: ${price:.2f}")
print(f"Number with leading zeros: {count:05d}")
print(f"Percentage: {0.856:.1%}")
```

---

15. Working with Files {#working-with-files}

Concept Explanation

Python can read from and write to files. The `open()` function opens a file, and files should be properly closed after use.

```
# Example 1: Writing to a File
print("Example 1: Writing to a File")

# Create and write to a file
with open("sample.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("This is Python basics notebook.\n")
    file.write("Learning file operations!\n")

print("File 'sample.txt' created successfully!")
```

```
# Example 2: Reading from a File
print("Example 2: Reading from a File")

# Read entire file
with open("sample.txt", "r") as file:
    content = file.read()
    print("File content:")
    print(content)
```

```
# Example 3: Reading File Line by Line
print("Example 3: Reading File Line by Line")

with open("sample.txt", "r") as file:
    print("Reading lines:")
    for line_num, line in enumerate(file, 1):
        print(f"  Line {line_num}: {line.strip()}")
```

```
# Example 4: Appending to a File
print("Example 4: Appending to a File")

# Append to existing file
with open("sample.txt", "a") as file:
    file.write("This line was appended.\n")

# Read and display
with open("sample.txt", "r") as file:
    print("Updated file content:")
    print(file.read())
```

---

16. Error Handling {#error-handling}

Concept Explanation

Error handling allows your program to handle exceptions gracefully without crashing. Use `try`, `except`, `else`, and `finally`.

```
# Example 1: Basic try-except
print("Example 1: Basic try-except")

try:
    number = int("abc")  # This will raise an error
except ValueError:
    print("Error: Cannot convert 'abc' to integer!")

print("Program continues after error handling.")
```

```
# Example 2: Multiple except blocks
print("Example 2: Multiple except blocks")

try:
    numbers = [1, 2, 3]
    print(numbers[5])  # IndexError
except ValueError:
    print("ValueError occurred!")
except IndexError:
    print("Error: Index out of range!")
except Exception as e:
    print(f"An error occurred: {e}")
```

```
# Example 3: try-except-else-finally
print("Example 3: try-except-else-finally")
```

```
try:
    x = 10
    y = 2
    result = x / y
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print(f"Division successful: {x} / {y} = {result}")
finally:
    print("Finally block: This always executes.")
```

```
# Example 4: Using finally for cleanup
print("Example 4: Using finally for resource cleanup")

try:
    file = open("sample.txt", "r")
    content = file.read()
    print(f"File read successfully, length: {len(content)} characters")
except FileNotFoundError:
    print("File not found!")
finally:
    if 'file' in locals():
        file.close()
        print("File closed.")
```

---

17. List Comprehensions {#list-comprehensions}

Concept Explanation

List comprehension is a concise way to create lists based on existing lists. It's more efficient and readable than using loops.

```
# Example 1: Basic list comprehension
print("Example 1: Basic list comprehension")

# Traditional way
squares_traditional = []
for i in range(1, 6):
    squares_traditional.append(i ** 2)
print(f"Squares (traditional): {squares_traditional}")

# List comprehension
squares_comp = [i ** 2 for i in range(1, 6)]
print(f"Squares (comprehension): {squares_comp}")
```

```
# Example 2: List comprehension with condition
print("Example 2: List comprehension with condition")

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Filter even numbers
evens = [x for x in numbers if x % 2 == 0]
print(f"Even numbers: {evens}")
```

```
# Filter and transform
even_squares = [x ** 2 for x in numbers if x % 2 == 0]
print(f"Squares of even numbers: {even_squares}")
```

```
# Example 3: Nested list comprehension
print("Example 3: Nested list comprehension")

# Create a multiplication table
matrix = [[i * j for j in range(1, 4)] for i in range(1, 4)]
print("3x3 Multiplication table:")
for row in matrix:
    print(f"  {row}")
```

```
# Example 4: String list comprehension
print("Example 4: String list comprehension")

words = ["hello", "world", "python"]

# Convert to uppercase
uppercase = [word.upper() for word in words]
print(f"Uppercase: {uppercase}")

# Get word lengths
lengths = [len(word) for word in words]
print(f"Word lengths: {lengths}")
```

---

18. Lambda Functions {#lambda-functions}

Concept Explanation

A **lambda function** is a small anonymous function defined with the `lambda` keyword. Lambdas are useful with `map()`, `filter()`, and `sorted()`.

```
# Example 1: Basic lambda function
print("Example 1: Basic lambda function")

# Lambda with regular function definition
add = lambda x, y: x + y
print(f"lambda x, y: x + y")
print(f"add(5, 3) = {add(5, 3)}")

square = lambda x: x ** 2
print(f"\nlambda x: x ** 2")
print(f"square(7) = {square(7)}")
```

```
# Example 2: Lambda with map()
print("Example 2: Lambda with map()")

numbers = [1, 2, 3, 4, 5]

# Apply function to all elements
doubled = list(map(lambda x: x * 2, numbers))
```

```
print(f"Original: {numbers}")
print(f"Doubled: {doubled}")

# Apply to multiple lists
list1 = [1, 2, 3]
list2 = [10, 20, 30]
sums = list(map(lambda x, y: x + y, list1, list2))
print(f"\nSum of {list1} and {list2}: {sums}")
```

```
# Example 3: Lambda with filter()
print("Example 3: Lambda with filter()")

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Filter even numbers
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(f"Original: {numbers}")
print(f"Even numbers: {evens}")

# Filter numbers greater than 5
greater_than_5 = list(filter(lambda x: x > 5, numbers))
print(f"Greater than 5: {greater_than_5}")
```

```
# Example 4: Lambda with sorted()
print("Example 4: Lambda with sorted()")

students = [("Alice", 85), ("Bob", 92), ("Charlie", 78), ("Diana", 88)]

# Sort by name
by_name = sorted(students, key=lambda x: x[0])
print("Sorted by name:")
for name, score in by_name:
    print(f"  {name}: {score}")

# Sort by score (descending)
by_score = sorted(students, key=lambda x: x[1], reverse=True)
print("\nSorted by score (highest first):")
for name, score in by_score:
    print(f"  {name}: {score}")
```

Summary and Key Takeaways

What We've Covered:

1. **Variables and Data Types** - Understanding Python's basic data types
2. **Numbers and Arithmetic** - Performing mathematical operations
3. **Strings** - Working with text data
4. **Lists** - Collections of mutable items
5. **Tuples** - Immutable sequences
6. **Dictionaries** - Key-value pair collections
7. **Sets** - Unique item collections
8. **Booleans** - Truth values and comparisons

9. **Conditionals** - Making decisions in code
10. **Loops** - Repeating code blocks
11. **Functions** - Reusable code blocks
12. **Input/Output** - Interacting with users
13. **Type Conversion** - Converting between types
14. **String Formatting** - Creating formatted strings
15. **File Operations** - Reading and writing files
16. **Error Handling** - Managing exceptions
17. **List Comprehensions** - Efficient list creation
18. **Lambda Functions** - Anonymous functions

Best Practices for Beginners:

- **Write clean code** with meaningful variable names
- **Use comments** to explain your logic
- **Follow PEP 8** Python style guide
- **Test your code** with different inputs
- **Handle errors gracefully** with try-except blocks
- **Practice regularly** with small projects
- **Read documentation** when learning new functions

Next Steps:

- Build small projects (calculator, to-do list, etc.)
- Learn libraries like NumPy, Pandas, and Matplotlib
- Explore web development with Flask or Django
- Study data structures and algorithms
- Contribute to open-source projects

Happy Learning! 🐍